

CS 577: Principles and Techniques of Data Science

Phase 2: Data Properties, Joins, Exploration, and Visualization

Fall 2026 | San Diego State University

Overview

This phase builds directly on your Phase 1 proposal. You are not building predictive models yet. The goal is to understand your dataset through structured exploration and simple visualization, to interrogate it using the five key data properties from lecture (structure, granularity, scope, temporality, and faithfulness), and to practice integrating a second data source. The dataset you clean in this phase becomes the **one** dataset you carry forward for the rest of the project.

By the end of this phase, you will have practiced the following:

- Revisiting your Phase 1 proposal and justifying whether your question or dataset should change
- Joining a second dataset to your own (your own second source, or a Google Trends CSV if you do not have one)
- Merging datasets using `pd.merge()`
- Auditing a dataset for structure, granularity, scope, temporality, and faithfulness
- Deliberately introducing errors into a copy of your data, then detecting them
- Reasoning about whether a dataset is ethically safe to send to an AI tool
- Writing multi-step Pandas queries with filtering, grouping, and aggregation
- Exploring data with Pandas operations
- Computing and interpreting a correlation coefficient
- Using `ipplot()` visualizations to answer concrete research questions
- Writing clear, evidence-backed interpretations of what your plots show

This is your introduction to visualization as a reasoning tool, not a design competition. Clarity and reasoning matter more than aesthetics.

Submission Requirements

Submit all of the following to the course submission portal:

- Your Jupyter Notebook (`.ipynb`)
- All dataset files used, including your second dataset (or Google Trends CSV)
- Your cleaned working dataset, saved as `phase2_working.csv` (see Part 2F)
- A short README section at the top of your notebook containing:

- Your original research question from Phase 1
- Your current research question and dataset, with a one-line note on whether either changed (details go in Part 0)
- Description of each dataset used
- Which second dataset you joined (your own or Google Trends), and an explanation of the join (column, join type, row counts before and after)
- A short summary of the cleaning steps you applied in Part 2F
- Final dataset shape (rows x columns)
- An AI use disclosure: which AI tool(s) you used anywhere in this phase, for what, and what data you sent. Write “None” if you used none.

Part 0: Revisiting Your Phase 1 Proposal

Real projects rarely survive first contact with the data unchanged. Before you do anything else, step back and ask whether your Phase 1 plan still holds. Changing your question or dataset is **not** penalized. An unjustified change, or an unjustified refusal to change when the evidence says you should, is.

In a markdown cell titled **Part 0: Proposal Revisit**, answer the following:

1. **What changed?** State which of the following applies: I changed my research question, I changed my dataset, I changed both, or I changed neither.
2. **On what basis?** Explain the evidence behind your decision in 3 to 5 sentences. Strong reasons are tied to the data itself. Examples include:
 - *Scope*: the data did not cover the population, region, or time period your question needs.
 - *Granularity*: the rows were too coarse (for example, state-level averages when you needed individual records) or too fine to answer the question directly.
 - *Temporality*: the data was outdated, or the date fields did not mean what you assumed.
 - *Faithfulness*: too many missing values, suspicious defaults, or duplicates to trust the results.
 - *Structure*: the data was nested or messy enough that it could not reasonably be made rectangular.
 - *Practical constraints*: fewer than 1,000 rows, a restrictive license, or instructor feedback from Phase 1.
3. **If you changed something:** state the new question and/or dataset, and write one or two sentences on what you gained and what you gave up.
4. **If you changed nothing:** do not write “it was fine.” Explain what you checked that convinced you the original plan still works. Part 2 below should back this up.

TIP

You may find it easier to write Part 0 last, after you have completed the data properties audit in Part 2. The audit is often where the reasons to change (or not change) show up.

Part 1: Dataset Preparation and Joins

1A. Choose Your Second Dataset (Required)

You must join your main dataset to a second dataset. Which one you use depends on what you already have:

- **If you already have a second dataset** (for example, one you identified in Phase 1, or a related table from the same source) that shares a column you can join on, **use it**. A shared ID, date, location, or category column all work. You do not need Google Trends.
- **If you do not have a second dataset**, download one from Google Trends as described below.

Either way, state in your README which option you chose and why the second dataset is relevant to your research question.

Google Trends option (only if you do not have a second dataset). To download your Google Trends CSV:

- Go to <https://trends.google.com>
- Search for a keyword relevant to your research question (for example, if your project is about housing prices, search for something like “home buying” or “mortgage rates”)
- Set the time range and region appropriate to your data
- Click the download icon (top right of the chart) to export as CSV
- Load the CSV into your notebook using `pd.read_csv()`

The Google Trends CSV contains a date column (for example, “Week”) and an interest over time column (numeric, 0 to 100). You will join this with a date or time column in your original dataset.

NOTE

The join is a required skill exercise. If you used Google Trends, you are joining to explore the relationship between public interest and your data, and you may decide not to keep the merged columns. If you used your own second dataset, the merged result may well become your working dataset. Either way, the decision you make in 1C is final (see below).

WATCH OUT: NO SECOND DATASET IS AS CLEAN AS IT LOOKS

Open the raw file in a text editor before loading it. Look for extra header lines above the real column names (use `skiprows` if so), placeholder values like `<1` or `N/A` that force a numeric column to be read as text, and dates that load as strings rather than datetimes. Google Trends exports are known for all three. Every one of these is a data property issue. Note which ones you hit, because you will need them in Part 2.

1B. Merge Requirements

You must use `pd.merge()` to combine your main dataset with your second dataset. You must clearly document the following in a markdown cell:

- The column(s) you joined on
- The type of join used (inner, left, right, or outer)
- Number of rows in each dataset before the join
- Number of rows in the merged result
- Why the row count changed, stayed the same, or might have dropped
- **New:** The granularity of each dataset before the join (for example, “one row per week” versus “one row per transaction”), and what you had to do to make the two line up (for example, converting daily dates to week start dates, or aggregating your data to weekly before joining)
- **New:** Whether your join key is unique in each table. Check this with `df[col].is_unique` or `df[col].duplicated().sum()` and explain what a non-unique key would do to your row count.

1C. Final Working Dataset Requirements

After the merge exercise, choose your working dataset: either your main dataset on its own, or the merged result if it meets the requirements below. You will audit and clean this dataset in Part 2, and the cleaned version is the dataset you use for everything that follows.

ONE DATASET, ALL THE WAY THROUGH

Once you clean your working dataset in Part 2F, you **must** use that same cleaned dataset for Parts 4 through 7 of this phase **and** for your Phase 3 models. You may not switch back to the raw file, swap in a different dataset, or quietly clean it a different way later. If you discover a new problem in Phase 3, fix it on top of `phase2_working.csv` and document the change. This is how real projects stay reproducible: one dataset, one documented trail of changes.

Your working dataset, both before and after cleaning, must:

- Contain at least 1,000 rows
- Contain at least 6 meaningful columns
- Include at least one categorical variable
- Include at least one numerical variable

Demonstrate the following in a clearly labeled notebook cell:

```
df.shape
df.info()
df.describe()
```

1D. Column Selection and Classification

Do **not** classify every column in your dataset. Real datasets often have dozens of columns, and most of them are irrelevant to any single question. Part of doing data science is deciding what to ignore.

Select between 5 and 8 columns that are directly relevant to your research question. Then, in one or two sentences, explain your selection: why these columns, and what kinds of columns you left out and why (for example, “I excluded the 14 internal ID and URL columns because they identify records but carry no information about price”).

Classify each selected column into one of the following variable types:

- **Quantitative Continuous:** numerical values that can take any value within a range (price, temperature, income)
- **Quantitative Discrete:** numerical values that are counted and cannot be subdivided (number of rooms, number of orders)
- **Qualitative Nominal:** categorical values with no natural order (city, genre, product category)
- **Qualitative Ordinal:** categorical values that have a meaningful order but unequal intervals (rating, education level, satisfaction score)

Create the following table in a markdown cell and fill it in for each of your 5 to 8 selected columns:

Column Name	dtype	Variable Type	Justification	Relevance to Question
example_col	float64	Quant. Continuous	Represents price, can take any decimal value	Price is the outcome my question is about

You must include a one-sentence justification and a one-sentence relevance note for each column. A dtype of `int64` does not automatically mean discrete, and `object` does not automatically mean nominal. Think about what the values actually represent. A ZIP code stored as `int64` is not a number you can average.

This classification will directly inform which visualizations are appropriate in Part 6. A histogram makes sense for quantitative continuous variables. A bar chart makes sense for nominal or ordinal ones. If you use the wrong plot type for a variable type, points will be deducted in the visualization section.

Part 2: Key Data Properties Audit

In lecture we covered five properties every data scientist should check before trusting a dataset. In this part you will answer each one for your working dataset. Every answer must be backed by **at least one code cell producing evidence** (an output you can point to) and a markdown cell of 2 to 4 full sentences explaining what the evidence shows. Answers with no code evidence will receive at most half credit.

Use the guiding questions below. You do not need to answer every bullet, but you must address the ones that apply to your data.

2A. Structure: What is the “shape” of the data file?

- What format did the data arrive in (CSV, TSV, Excel, JSON, XML, SQL, log file)? Did you have to un-nest or parse anything to make it rectangular?
- Is there a primary key (a column or set of columns that uniquely identifies each row)? Prove it with `is_unique` or `duplicated()`.
- Does your data reference other data through a foreign key? Could it be joined to anything else?
- How are fields encoded? Are numbers stored as strings, or dates stored as text?

2B. Granularity: How fine or coarse is each record?

- What does one row represent? (a person, a transaction, a city, a week, a group of users)
- Are all rows at the same level? Look for summary or rollup rows hiding in the data, such as a “Total” or “All” row.
- If the data is aggregated, how was it aggregated (sums, averages, sampling)? Compare this to the granularity of your second dataset.

2C. Scope: How complete is the data?

- Does the data cover the population, region, and time period your question is about?
- Is it too broad, so that you need to filter? If you filter, how many rows remain, and is the remaining sample still representative?
- What is the sampling frame, and how does it differ from your population of interest? (Recall the Literary Digest.)

2D. Temporality: How is the data situated in time?

- What is the date range? Show `min()` and `max()` of your date column.
- What does the timestamp actually mean: when the event happened, when it was recorded, or when it was copied into the database?
- Are there time zone or format issues (is 07/08/09 July 8 or August 7)?
- Are there suspicious dates such as January 1, 1970 or January 1, 1900? Is there periodicity (weekly, seasonal, daily cycles)?
- If your dataset has no time column at all, say so and explain what that limits.

2E. Faithfulness: Do you trust this data?

- Run `df.isnull().sum()` and report missing values per column. (This was an optional extension in earlier versions. It is now required.)
- Look for sentinel or default values that are pretending not to be missing: 0, -1, 999, empty strings, “N/A”, or 1970 and 1900 dates. `value_counts()` and `describe()` are your friends here.

- Check for impossible values (negative counts, future dates for past events, ages over 130) and violated dependencies (age and birth year that disagree).
- Check for duplicate rows with `df.duplicated().sum()`.
- Look for signs of hand entry: inconsistent spellings or capitalization of the same category (“San Diego”, “san diego”, “SD”).
- State what you would do about each problem (drop, impute, correct, or leave and document), and what that choice might do to your sample.

2F. Clean and Lock In Your Working Dataset

Now act on what you found. Apply the fixes you proposed in 2A through 2E (dropping, imputing, correcting types, standardizing spellings, removing duplicates, filtering to your scope, and so on) and produce your final cleaned dataset.

- Do all cleaning in code, in clearly labeled cells, so every step can be rerun from the raw file. No hand edits in Excel.
- In a markdown cell, list each cleaning step, the reason for it (tied to one of the five properties), and how many rows or values it affected.
- Show `df.shape` before and after cleaning, and confirm the cleaned dataset still meets the 1C requirements.
- Save the result and submit it with your notebook:

```
df_clean.to_csv('phase2_working.csv', index=False)
```

This file is your dataset for the rest of the project. Parts 4 through 7 and Phase 3 must all start from it.

Part 3: Faithfulness Stress Test (Break It on Purpose)

Most data errors are not made by villains. They are made by ordinary, well-meaning steps: a join, a quick trip through Excel, or asking an AI to “just convert this to JSON.” The best way to learn to catch these errors is to cause them yourself, on purpose, in a safe copy.

GROUND RULES

- Always work on a copy. Never overwrite your original data. For example:
`df_test = df.sample(50, random_state=577).copy()`
- Your analysis in Parts 4 through 7 must use your cleaned working dataset from Part 2F, never the broken test copy.
- Before you send *any* data to an AI tool, you must complete the AI Ethics Clearance in 3C. If your data fails the clearance, you may not send it. Use the synthetic data option instead.

3A. Choose Your Corruption Pathways

Complete **at least two** pathways, drawn from **at least two different categories** below. Ideas are provided to get you started. You may invent your own as long as it reflects something that could

realistically happen.

Category 1: Combining data

- *Granularity mismatch*: join your data directly to your second dataset at mismatched granularity (for example, daily or per-transaction rows against weekly Google Trends rows) without aligning them first. Count how many rows fail to match.
- *Non-unique key*: join two tables on a key that repeats in both. Watch the row count explode (a many-to-many join).
- *Type mismatch*: join on a date column that is a string in one table and a datetime in the other. See whether anything matches at all.
- *Outer join side effects*: perform an outer join and check what happened to your integer columns. (Hint: NaN is a float.)
- *Concatenation drift*: split your data in two, rename one column slightly in one half (“Price” versus “price”), and `pd.concat()` them back together.

Category 2: Format conversion by hand or with common tools

- Open your CSV in Excel or Google Sheets, save it, and reload it. Check for lost leading zeros (ZIP codes, IDs), long IDs turned into scientific notation, and dates that were silently reformatted.
- Save with a different encoding (for example, Latin-1 instead of UTF-8) and reload. Look for garbled accented characters.
- Introduce a field containing a comma without quoting it, and see what `pd.read_csv()` does to that row.
- Convert a timestamp column to a different time zone and back, and check whether any values shifted.

Category 3: AI-assisted conversion (requires 3C clearance first)

- Give an AI tool 30 to 50 rows of your CSV and ask it to convert them to JSON (or JSON to CSV). Load the result back into Pandas and compare it to the original.
- Ask an AI tool to “clean” or “standardize” those rows. Find every change it made, especially the ones it did not tell you about.
- Ask an AI tool to reformat a date column or translate category labels. Check for invented categories or reordered dates.

Common AI conversion errors to look for include dropped rows, truncated output (“and 20 more rows...”), missing values silently filled in, numbers rounded or reformatted, dates changed to a different convention, spellings “helpfully” corrected, and entirely invented values. These are faithfulness failures, and they are easy to miss because the output *looks* clean.

3B. Detect and Document

For each pathway, detect the damage using code, not eyeballing. Useful tools include:

```
df_test.shape, df_broken.shape          # did rows go missing or multiply?
df_test.dtypes == df_broken.dtypes      # did any types change?
df_broken.isnull().sum()                 # did missing values appear?
df_broken.duplicated().sum()             # did duplicates appear?
set(df_test['key']) - set(df_broken['key']) # which records vanished?
df_test.reset_index(drop=True).compare(df_broken.reset_index(drop=True))
```

Note that `.compare()` only works when both frames have the same shape and column labels. If they do not, that mismatch is itself a finding.

For each pathway, write a markdown cell that answers:

1. What did you do? (Include the exact AI prompt if you used one.)
2. What broke, and how many rows or values were affected?
3. How did you detect it? Point to the specific code output.
4. Which of the five data properties does this error violate?
5. If you had not caught this, how would it have affected your EDA in this phase or your models in Phase 3?

3C. AI Ethics Clearance (Required Before Any AI Pathway)

“It is on the internet” and “it is for a class project” are not ethical justifications. When you paste data into an AI tool, you are sending it to a third party’s servers, where it may be logged, retained, reviewed by people, or used for training depending on the tool and your settings. You are responsible for knowing which.

If you choose a Category 3 pathway, include a markdown cell titled **AI Ethics Clearance** that answers each question below in one to three sentences. If you did not use AI in Part 3, write one sentence stating that, and briefly explain whether your data *would* have passed this clearance.

1. **Personal information:** Does the data contain names, emails, phone numbers, addresses, account IDs, or other personally identifiable information (PII)? Does it contain *quasi-identifiers*, meaning columns that are harmless alone but identifying in combination (for example, ZIP code plus birth date plus gender)?
2. **Sensitive categories:** Does it include health, financial, criminal justice, immigration, or student records, or data about minors? Anonymized data in these categories still deserves extra caution because it can sometimes be re-identified.
3. **Consent and context:** Did the people in the data agree to its collection? Would they reasonably expect it to be fed to an AI system? Scraped social media posts are publicly visible, but that is not the same as consent to every use.
4. **License and terms of use:** Does the dataset’s license or terms of service permit sharing it with a third-party service? Check the source page (Kaggle, data.gov, the original publisher, and so on).

5. **Tool and retention:** Which AI tool did you use? Does it retain inputs or use them for training under your settings? Is it a tool your institution has approved for this kind of data?
6. **Minimum necessary:** Did you send only what was needed (a small sample of rows and only the relevant columns) rather than the whole dataset?
7. **Verdict:** Based on the above, state clearly whether the data is safe to send, and why.

IF YOUR DATA DOES NOT PASS

You can still do an AI pathway. Create 30 to 50 rows of *synthetic* data with the same column names and types as your real data (you can generate it with NumPy or Pandas), and run the experiment on that instead. Note in your clearance cell that you did so. Choosing not to send real data is a correct and full-credit answer when the clearance says no. As a point of reference, a Google Trends export is public, aggregated, and contains no individuals, so it is a good example of data that usually passes. A dataset of patient records or student grades is a good example of data that does not.

Part 4: Pandas Query

You are required to write at least one meaningful query against your dataset using Pandas. It must combine at least two operations (for example, a filter followed by a grouped aggregation), not just `df.head()` or a single column lookup.

Option A: Boolean filtering with groupby

```
result = (df[df['numeric_col'] > 0]
          .groupby('category_col')['numeric_col']
          .mean()
          .sort_values(ascending=False))
print(result)
```

Option B: Using `df.query()` with agg

```
result = (df.query('year_col >= 2020')
          .groupby('category_col')
          .agg(count=('numeric_col', 'size'), avg_val=('numeric_col', 'mean'))
          .sort_values('count', ascending=False))
print(result)
```

Your query cell must include a markdown explanation of:

- What question the query is answering
- What each step of the chain does (the filter, the grouping, and the aggregation)
- What the output shows
- How this connects to your overall research question

Part 5: Correlation Analysis

You must compute and interpret the Pearson correlation coefficient between at least two numerical columns in your dataset. This is not a visualization requirement; it is a statistical reasoning requirement.

Use the following structure:

```
correlation = df['numeric_col_1'].corr(df['numeric_col_2'])
print(f'Correlation between X and Y: {correlation:.4f}')
```

You may optionally compute a full correlation matrix:

```
df[['col1', 'col2', 'col3']].corr()
```

In a markdown cell, write a 3 to 5 sentence interpretation addressing:

- What the numeric value means (positive, negative, magnitude)
- Whether the correlation is strong, moderate, or weak
- What this suggests about the relationship between the two variables
- Any caveats (correlation does not imply causation; possible confounders)
- **New:** Whether any faithfulness issue from Part 2 could be distorting this number (for example, sentinel values like 999 or 0 left in a column can drag a correlation up or down)

Part 6: Exploratory Data Analysis (EDA) Questions

You must answer at least four specific EDA questions using visualization. Each question must:

- Be clearly stated in a markdown cell before the code
- Use Pandas operations to prepare the data
- Include one `imshow()` visualization
- Include a written interpretation of 3 to 5 full sentences in a markdown cell after the plot

You may not submit screenshots without a written explanation.

Required Visualization 1: Category Counts (Bar Chart)

Example question: Which category appears most frequently in the dataset?

```
df['category_col'].value_counts().plot(kind='bar', title='Category Counts')
```

Required Visualization 2: Grouped Aggregation (Bar Chart)

Example question: Which group has the highest average value for the target variable?

```
df.groupby('group_col')['numeric_col'].mean().plot(kind='bar', title='Average by Group')
```

Required Visualization 3: Distribution (Histogram)

Example questions: Is the distribution of this variable symmetric or skewed? Are there potential outliers?

```
df['numeric_col'].plot(kind='hist', bins=30, title='Distribution of Numeric Variable')
```

Required Visualization 4: Trend or Top-N Comparison

If your dataset contains a time variable:

```
df.groupby('year_col')['numeric_col'].mean().plot(kind='line', title='Trend Over Time')
```

If your dataset does not contain time data:

```
df.sort_values('numeric_col', ascending=False).head(10).plot(kind='bar', title='Top 10 by Numeric Variable')
```

Part 7: Visualization Interpretation Guide

For every visualization in Part 6, you must answer the following questions in full sentences in a markdown cell immediately after the plot:

- What question are you asking?
- What method did you use to create the visualization?
- What does the visualization show?
- What insight can you draw from it, and how does it connect to your research question?
- **New:** Does the plot reveal anything about the data properties from Part 2? For example, a spike at exactly 0 or 999 in a histogram suggests a sentinel value (faithfulness), a sudden jump in a trend line may reflect a change in how data was recorded (temporality), and a category with only a handful of rows may reveal a scope gap. If nothing stands out, say so in one sentence.

Clarity and reasoning matter more than aesthetics. A plain bar chart with a sharp interpretation will score higher than a decorative one with a vague description.

Grading Criteria

Criterion	Points
Part 0: Proposal revisit, with an evidence-based justification for changing or keeping the question and dataset	5
Part 1: Correct implementation and documentation of the second-dataset join using <code>pd.merge()</code> , including granularity alignment and key uniqueness	20
Part 1: Dataset meets size and structure requirements (1,000+ rows, 6+ columns, 1 categorical, 1 numerical); 5 to 8 relevant columns selected, justified, and correctly classified	10
Part 2: Key data properties audit (structure, granularity, scope, temporality, faithfulness), each backed by code evidence	15
Part 3: Faithfulness stress test (two pathways, two categories, code-based detection) and AI Ethics Clearance	10
Parts 4 and 6: Correct use of Pandas operations, including a meaningful multi-step Pandas query	10
Part 6: Quality and clarity of EDA questions (4 required)	10
Part 6: Appropriate use of required visualizations (bar, grouped bar, histogram, trend or top-N) using <code>iplot()</code>	10
Parts 5 and 7: Depth and clarity of written interpretations and correlation analysis	10
Total	100

Optional Extensions (Not Graded)

The following are not required for this phase but will make your Phase 3 modeling work significantly easier if completed now:

- Identifying and documenting exactly which rows were lost during the join
- Visualizing the correlation matrix as a heatmap
- Writing a small reusable validation function (for example, `check_integrity(df)`) that runs your Part 3 detection checks in one call, so you can rerun it every time you transform your data in Phase 3
- Running a third corruption pathway from the category you did not use

Strong exploratory analysis here directly informs the features you will use in your predictive models in Phase 3. The time you put in now is not wasted.